

Câu 1: Lập trình cơ bản C#

BÀI TẬP:

```
static List<int> values = new List<int>
    { 0, 1, 2, 4, 6, 8, 9, 10, 15, 21, 25, 30, 36, 49, 64 };
static List<int> sortedA = new List<int> { -4, -1, 0, 2, 5,
    8, 11, 14 };
static List<int> sortedB = new List<int> { 1, 2, 4, 7, 9, 11,
    15 };
```

(1a) Viết hàm `IsEvenPositive(int n)`: trả về `true` nếu `n` là số chẵn và lớn hơn 0.

```
1 static bool IsEvenPositive(int n)
2 {
3     // TODO: CODE HERE
4 }
```

(1b) Viết hàm `IsMultipleOfThree(int n)`: kiểm tra `n` có chia hết cho 3 hay không.

```
1 static bool IsMultipleOfThree(int n)
2 {
3     // TODO: CODE HERE
4 }
```

(1c) Viết hàm `CountEvenNumbers(List<int> source)`: đếm bao nhiêu số chẵn trong danh sách.

```
1 static int CountEvenNumbers(List<int> source)
2 {
3     // TODO: CODE HERE
4 }
```

(1d) Viết hàm `GetLargestMultipleOfThree(List<int> source)`: trả về số lớn nhất chia hết cho 3; nếu không có thì trả về -1.

```
1 static int GetLargestMultipleOfThree(List<int> source)
2 {
3     // TODO: CODE HERE
4 }
```

(1e) Viết hàm `HasPairWithSum(List<int> sorted, int target)` trên danh sách đã sắp xếp tăng dần.

```
1 static bool HasPairWithSum(List<int> sorted, int target)
2 {
3     // TODO: CODE HERE
4 }
```

(1f) Viết hàm `GetCommonValues(List<int> a, List<int> b)`: trả về danh sách các phần tử chung của hai danh sách đã sắp xếp tăng dần.

```

1 static List<int> GetCommonValues(List<int> a, List<int> b)
2 {
3     // TODO: CODE HERE
4 }

```

Câu 2:

```

public class Course
{
    public long Id { get; set; }
    public string? Title { get; set; }
    public int Credits { get; set; }
}

static List<Course> courses = new List<Course>
{
    new Course { Id = 1, Title = "Lap trình C#", Credits = 3 },
    new Course { Id = 2, Title = "Co so du lieu", Credits = 4 },
    new Course { Id = 3, Title = null, Credits = 2 },
};

```

(2a) Khai báo một delegate tên `FormatCourse` nhận vào một `Course` và trả về chuỗi theo mẫu "Id - Title".

```

1 // TODO: CODE HERE

```

(2b) Viết `Func<Course, string>` để lấy tên học phần; nếu Title rỗng thì trả về "(chưa có tên)".

```

1 // TODO: CODE HERE

```

(2c) Viết `Action<Course>` để in thông tin học phần ra màn hình theo mẫu [Id] Title - Credits.

```

1 // TODO: CODE HERE

```

(2d) Viết `Predicate<Course>` để lọc các học phần có số tín chỉ từ 3 trở lên, rồi dùng với `FindAll`.

```

1 // TODO: CODE HERE

```

Câu 3: Book và IBooksRepository

```
public class Book
{
    public long Id { get; set; }
    public string? Title { get; set; }
    public decimal Price { get; set; }
}
```

(3a) Khai báo interface IBooksRepository với 5 phương thức: GetAll, GetById, Add, GetBooksCheaperThan, GetTopExpensiveBooks.

```
1 public interface IBooksRepository
2 {
3     // TODO: CODE HERE
4 }
```

(3b) Viết constructor của BooksRepository, khởi tạo ít nhất 3 cuốn sách mẫu.

```
1 public class BooksRepository : IBooksRepository
2 {
3     private List<Book> _books = new();
4
5     public BooksRepository()
6     {
7         // TODO: them du lieu mau
8     }
9
10    public IList<Book> GetAll() => _books;
11    public Book? GetById(long id) => _books.FirstOrDefault(b =>
        b.Id == id);
12    public Book Add(Book book)
13    {
14        _books.Add(book);
15        return book;
16    }
17
18    public IReadOnlyList<Book> GetBooksCheaperThan(decimal
        maxPrice)
19    {
20        // TODO: CODE HERE
21    }
22
23    public IReadOnlyList<Book> GetTopExpensiveBooks(int n)
24    {
25        // TODO: CODE HERE
26    }
27 }
```

Câu 4:

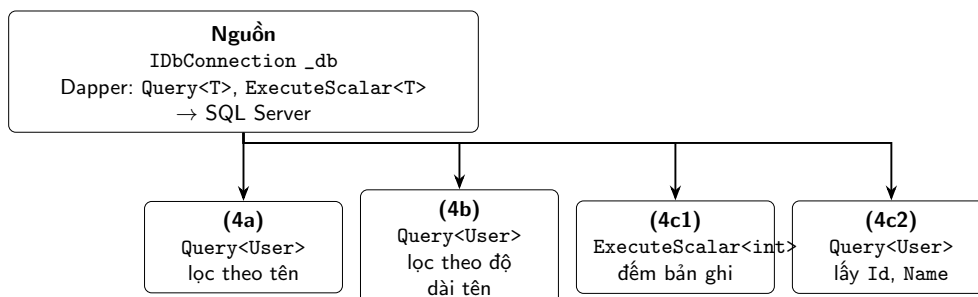
```
public class User
{
    public long Id { get; set; }
    public string? Name { get; set; }
}

public class UsersDapperRepository
{
    private readonly IDbConnection _db;
    public UsersDapperRepository(IDbConnection db) { _db = db; }

    public IList<User> GetAll()
        => _db.Query<User>("SELECT * FROM Users").ToList();
}
```

Yêu cầu chung: Bổ sung các phương thức mới vào đúng lớp UsersDapperRepository. Tất cả các phương thức phải truy vấn trực tiếp qua _db bằng Dapper.

Sơ đồ luồng (Câu 4): Hai ý đầu trả về danh sách User; ý cuối gồm một phương thức đếm và một phương thức trả về danh sách User chỉ gồm Id, Name.



(4a) Viết FindNamesEndingWith(string name): trả về danh sách user có tên kết thúc bằng name.

```
1 public IReadOnlyList<User> FindNamesEndingWith(string name)
2 {
3     // TODO: CODE HERE
4 }
```

(4b) Viết GetUsersWithShortName(int maxLength): trả về danh sách user có tên ngắn hơn hoặc bằng maxLength ký tự.

```
1 public IReadOnlyList<User> GetUsersWithShortName(int maxLength)
2 {
3     // TODO: CODE HERE
4 }
```

(4c) Viết hai phương thức: CountNamesContaining(string keyword) và GetIdAndNameList().

```
1 public int CountNamesContaining(string keyword)
```

```

2 {
3     // TODO: CODE HERE
4 }
5
6 public List<User> GetIdAndNameList()
7 {
8     // TODO: CODE HERE
9 }

```

Câu 5:

```

public class BooksService
{
    public IReadOnlyList<Book> GetBooksCheaperThan(decimal
        maxPrice)
    {
        // Gia su da duoc cai dat
        throw new NotImplementedException();
    }
}

```

(5) Thêm action `GetCheapBooks` vào `BooksController` cho đường dẫn GET `/books/cheap?maxPrice=...`. Nếu `maxPrice <= 0` thì trả về `BadRequest`; nếu hợp lệ thì gọi `_service.GetBooksCheaperThan(maxPrice)` và trả về `Ok`.

```

1 [ApiController]
2 [Route("[controller]")]
3 public class BooksController : ControllerBase
4 {
5     private readonly BooksService _service;
6     public BooksController(BooksService service) { _service =
        service; }
7
8     [HttpGet("cheap")]
9     public IActionResult GetCheapBooks([FromQuery] decimal
        maxPrice)
10    {
11        // TODO: CODE HERE
12    }
13 }

```